



Proiect la disciplina Sisteme de Conducere in Robotica

Brat Robotic PUMA

Autor: Student Laurentiu-Daniel CONDREA
Programul de studii: Robotică
Grupa 4LF811

Coordonator: Prof. univ. dr. ing. Claudiu POZNA

2025

Cuprins

1.Introducere	3
2.Traietorie Pick-and-Place	3
3.Metode.....	4
3.1Controlul cu regulator PID	5
3.1Controlul clasic	7
3.1Controlul dinamic invers	9
3.1Controlul fantoma	14
3.1Controlul in Spatiul Cartezian.....	17
4.Bibliografie	19

1. INTRODUCERE

Scopul proiectului de an la disciplina *Sisteme de Conducere in Robotica* este de a simula un brat robotic de tip PUMA (proiectat in CATIA) in Matlab Simulink utilizand diferite metode, analizand performantele fiecareia.

2. TRAIECTORIE PICK-AND-PLACE

Se foloseste o traiectorie de tip pick-and-place:

- Q_initial (pozitia cuplelor initiala)
- Q_final (pozitia cuplelor finala)

```
Q_initial=[0;0;0]; % Punct de plecare
Q_final=[1;2;3]; % Punct destinatie
[qt,viteza,acceleratie,t]=PickandPlacepuma(Q_initial,Q_final,4)
qt1=[t',qt(1,:)']
qt2=[t',qt(2,:)']
qt3=[t',qt(3,:)']

v1=[t',viteza(1,:)']
v2=[t',viteza(2,:)']
v3=[t',viteza(3,:)']

acc1=[t',acceleratie(1,:)']
acc2=[t',acceleratie(2,:)']
acc3=[t',acceleratie(3,:)']
```

Fig 2.1 Pozitia,viteza si acceleratia pentru fiecare cupla in raport cu timpul

Si rezulta, cu ajutorul functiei polinomiale, pozitia, viteza si acceleratia, in timp de 4 secunde:

```
function [qt,viteza,acceleratie,t] =PickandPlacepuma(Q_initial,Q_final,delta)
t=linspace(0,delta);
%Coeficienti functie polinomiale
a0=Q_initial;
a1=0;
a2=3*(Q_final-Q_initial)/(delta^2);
a3=-2*(Q_final-Q_initial)/(delta^3);

qt=a3*t.^3+a2*t.^2+a1*t+a0; %pozitie
viteza = 3*a3*t.^2 + 2*a2*t + a1;
acceleratie = 6*a3*t + 2*a2;
```

Fig 2.1 Functia polinomiale

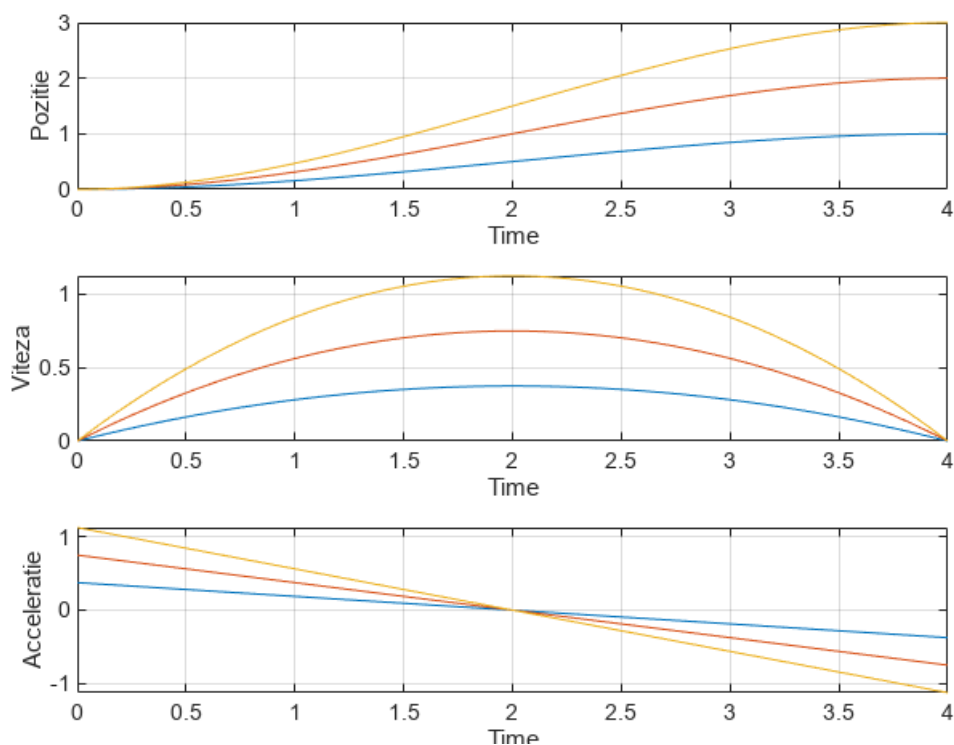


Fig 2.2 Grafice pozitie, viteza, acceleratie pentru fiecare cupla(albastru-cupla1, rosu-cupla2,galben-cupla3)

3. METODE

1.Controlul cu regulator PID
2.Controlul clasic
3.Controlul dinamic invers
4.Modelul fantoma
5.Controlul in Spatiul Cartezian

3.1 CONTROLUL CU REGULATOR PID

În această metodă se simulează robotul cu ajutorul unui regulator PID.

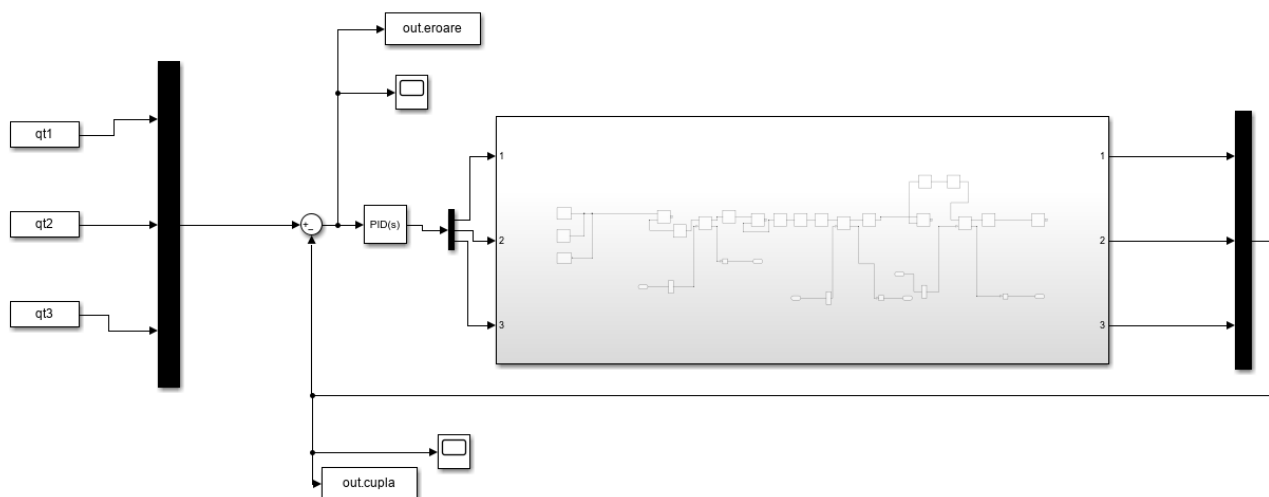


Figura 3.1.1 Schema block Simulink

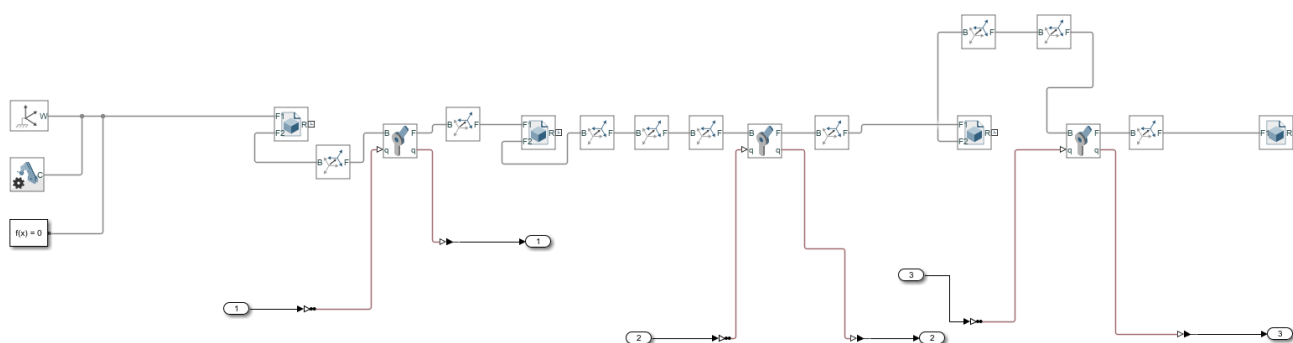


Figura 3.1.2 Modelul Robotului

Parametrii regulatorul PID: $K_p=100$, deoarece robotul nu își completează mișcarea în cele 4 secunde.

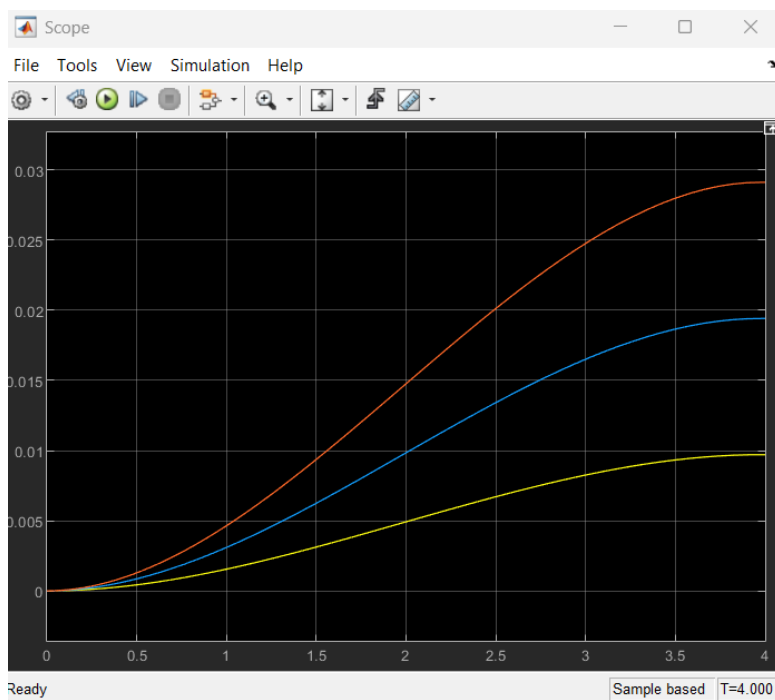


Fig 3.1.3 Eroarea

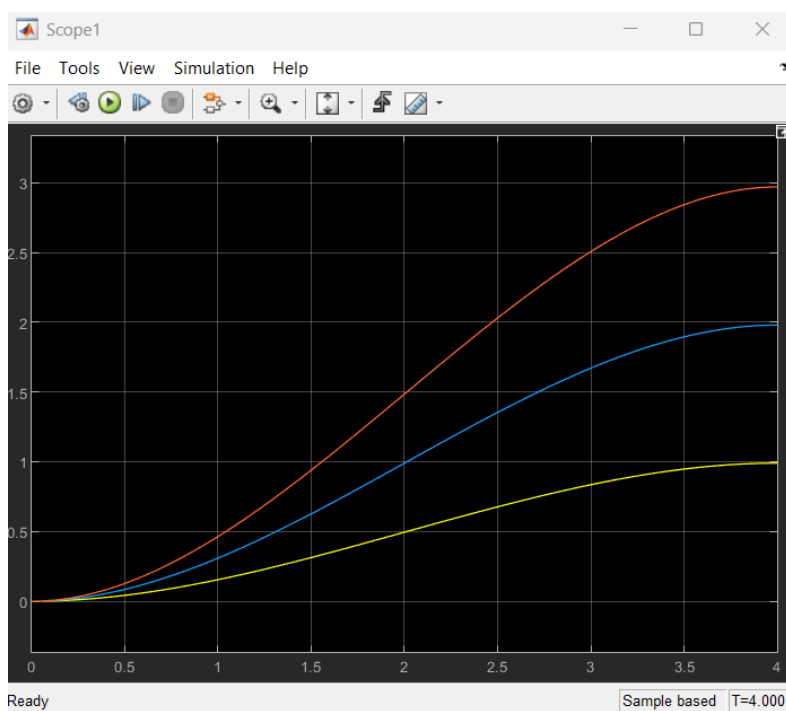


Fig 3.1.4 Pozitiile cuplelor

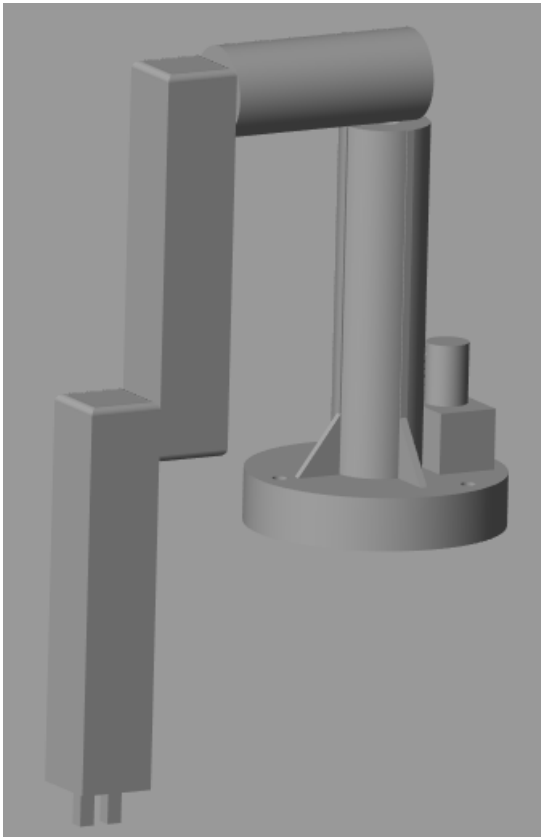


Fig 3.1.5 Robotul PUMA in (0,0,0)



Fig 3.1.6 Robotul PUMA in (1,2,3)

3.2 CONTROLUL CLASIC

Controlul clasic este cea de-a doua schema. Singurul lucru care se modifica fata de schema precedenta este prezenta moto-reductoarelor. Cu ajutorul Dinamicii Directe, se scot momentele fiecărei cuple si se baga in moto-reductoare (Momentele = perturbatii). Parametrii regulatorului PID sunt *default*.

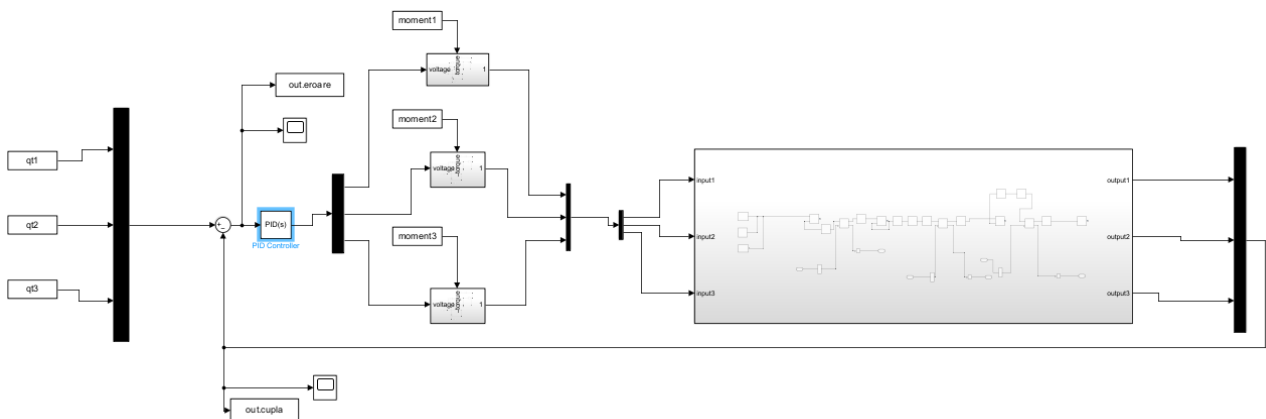


Fig 3.2.1 Schema block Simulink

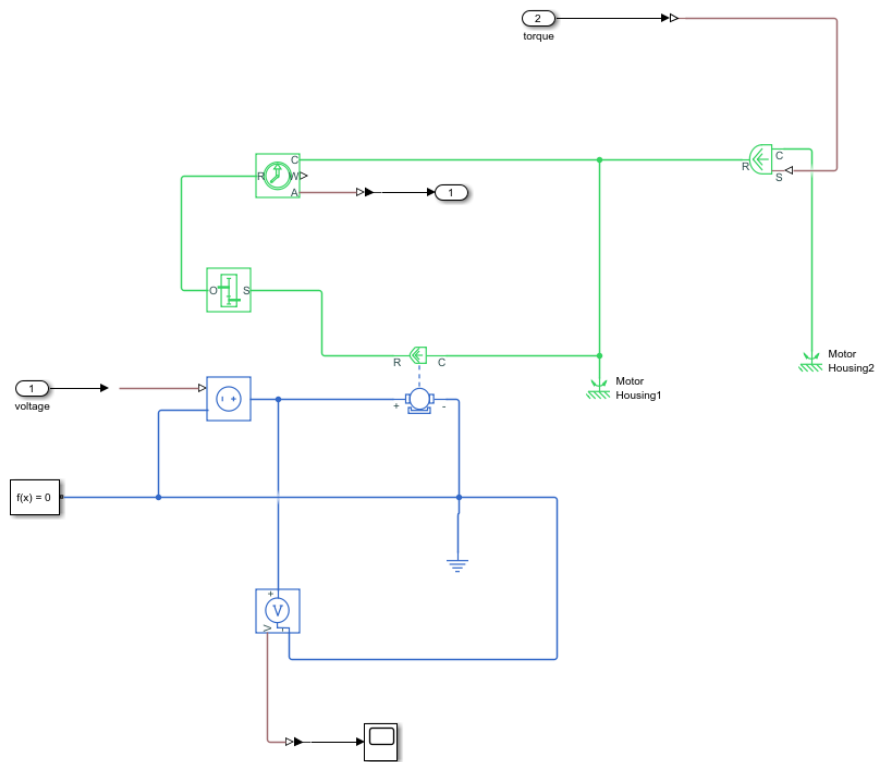


Fig 3.2.2 Moto-reductorul Simulink

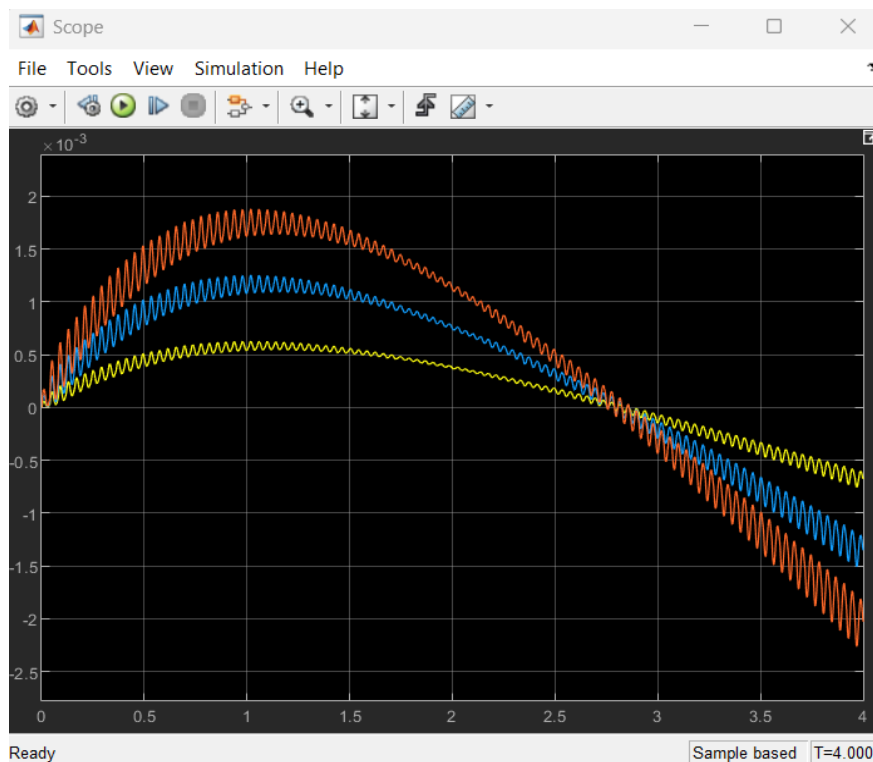


Fig 3.2.3 Eroarea

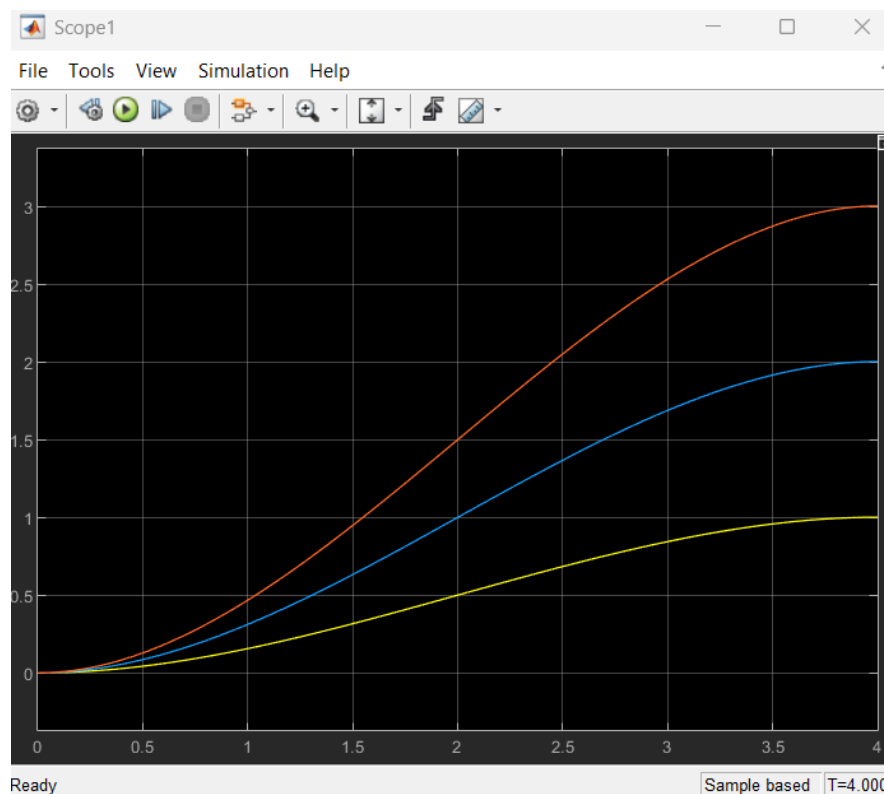


Fig 3.2.4 Pozitia cuplelor

3.4 CONTROLUL DINAMIC INVERS

În controlul dinamic invers este nevoie de matricile Coriolis, centrifuga, de inerție și gravitațională. Pentru a le putea integra în Simulink s-a folosit block-ul *MatlabFunction*.

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q})[\dot{\mathbf{q}}^2] + \mathbf{B}(\mathbf{q})[\dot{\mathbf{q}}\dot{\mathbf{q}}] + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}$$

Fig 3.4.1 Modelul dinamic

Unde: $\mathbf{M}(\mathbf{q})$ este matricea de inerție

$\mathbf{B}(\mathbf{q})$ este matricea Coriolis

$\mathbf{C}(\mathbf{q})$ este matricea Centrifuga

$\mathbf{G}(\mathbf{q})$ este matricea gravitațională

Cu ajutorul metodei Lagrange-Euler se determină aceste matrici.

$$\mathbf{M}(\mathbf{q}) = \sum_{i=1}^n \left(m_i \mathbf{J}_{vi}^T \mathbf{J}_{vi} + \mathbf{J}_{\omega i}^T \mathbf{I}_{Ci} \mathbf{J}_{\omega i} \right)$$

Fig 3.4.2 Matricea de inerție

$$\mathbf{C}(\mathbf{q})[\dot{\mathbf{q}}^2] = \begin{bmatrix} b_{111} & b_{122} & \dots & b_{1nn} \\ b_{211} & b_{222} & \dots & b_{2nn} \\ \vdots & \vdots & \dots & \vdots \\ b_{n11} & b_{n22} & \dots & b_{nnn} \end{bmatrix} \begin{bmatrix} q_1^2 \\ q_2^2 \\ \vdots \\ q_n^2 \end{bmatrix}$$

Fig 3.4.3 Matricea Centrifuga

$$\mathbf{B}(\mathbf{q})[\dot{\mathbf{q}}\dot{\mathbf{q}}] = \begin{bmatrix} 2b_{112} & 2b_{123} & \dots & 2b_{1n-1n} \\ 2b_{212} & 2b_{223} & \dots & 2b_{2n-1n} \\ \vdots & \vdots & \dots & \vdots \\ 2b_{n12} & 2b_{n23} & \dots & 2b_{nn-1n} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \dot{q}_2 \\ \dot{q}_1 \dot{q}_3 \\ \vdots \\ \dot{q}_{n-1} \dot{q}_n \end{bmatrix}$$

Fig 3.4.4 Matricea Coriolis

$$\mathbf{G}(\mathbf{q}) = - \begin{bmatrix} \mathbf{J}_{v1}^T & \mathbf{J}_{v2}^T & \dots & \mathbf{J}_{vn}^T \end{bmatrix} \cdot \begin{bmatrix} m_1 \mathbf{g} \\ m_2 \mathbf{g} \\ \vdots \\ m_n \mathbf{g} \end{bmatrix}$$

Fig 3.4.5 Matricea Gravitatională

Rezulta momentele, pentru fiecare cupla, care se integreaza in modelul robotului.

```
function G = matrice_gravitational(q)
g=9.81;
xc_1 = -0.36; yc_1 = 70.689; zc_1 = 0;
Ixx_1 = 0.0007006; Iyy_1 = 0.0001766; Izz_1 = 0.0006981; m_1 = 0.381;

xc_2 = 0; yc_2 = 127.75; zc_2 = 0;
Ixx_2 = 0.003; Iyy_2 = 0.0002608; Izz_2 = 0.003; m_2 = 0.618;

xc_3 = -0.15; yc_3 = 124.842; zc_3 = 0;
Ixx_3 = 0.004; Iyy_3 = 0.000264; Izz_3 = 0.004; m_3 = 0.633;

x_3 = 0; y_3 = -185.3; z_3 = 171.5;
x_2 = 0; y_2 = 30; z_2 = 121.5;

g_1=0;
g_2=-m_2*(xc_2*cos(q(2,:))+zc_2*sin(q(2,:)))*g-m_3*(xc_3*cos(q(2,:)+q(3,:))+zc_3*sin(q(2,:)+q(3,:))+x_3*cos(q(2,:))+z_3*sin(q(2,:)))*g;
g_3=-m_3*(xc_3*cos(q(2,:)+q(3,:))+zc_3*sin(q(2,:)+q(3,:)))*g;
G=[g_1;g_2;g_3];
```

Fig 3.4.6 MatlabFunction matrice gravitationala

In Fig 3.4.6 este implementarea matricei gravitationale, unde:

xc, yc, zc = centre de masa (rosu)

m = masa obiectului (verde)

Ixx, Iyy, Izz = matricea de inertie (albastru)

X2, y2 =distanța dintre cuple

Aceste date au fost preluate pentru fiecare brat in parte al robotului, din CATIA:

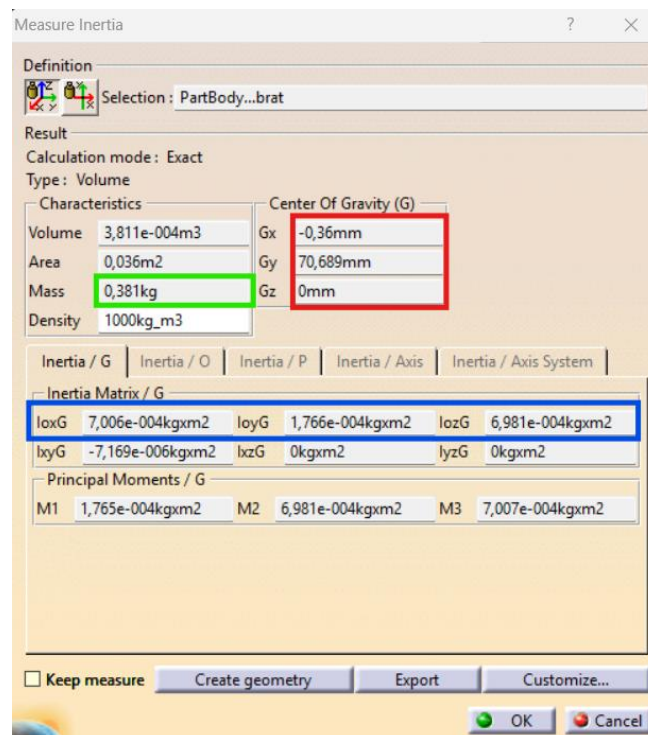


Fig 3.4.7 Date CATIA

Asemanator sunt realizate si restul matricilor, folosind ecuatiile specific.

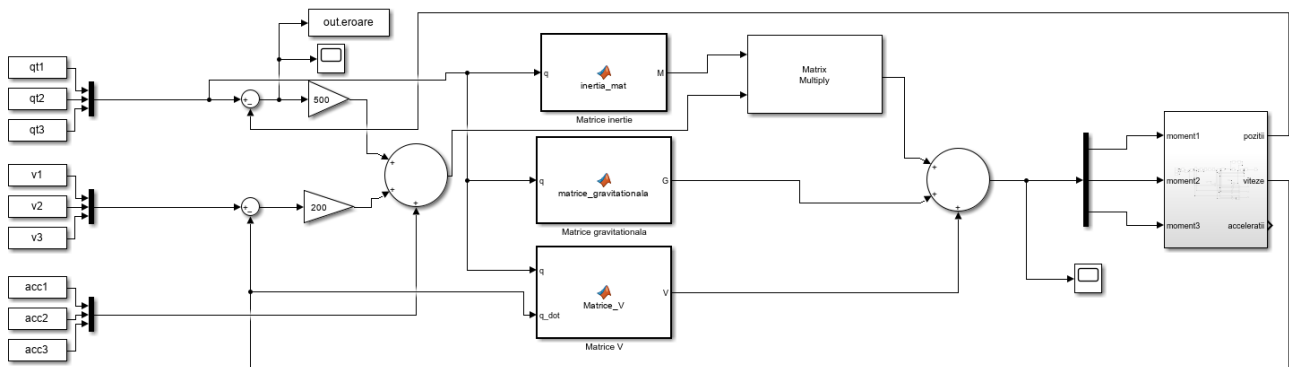


Fig 3.4.8 Schema block Simulink

Valorile la PID sunt : $K_p=500$ si $K_i=200$.

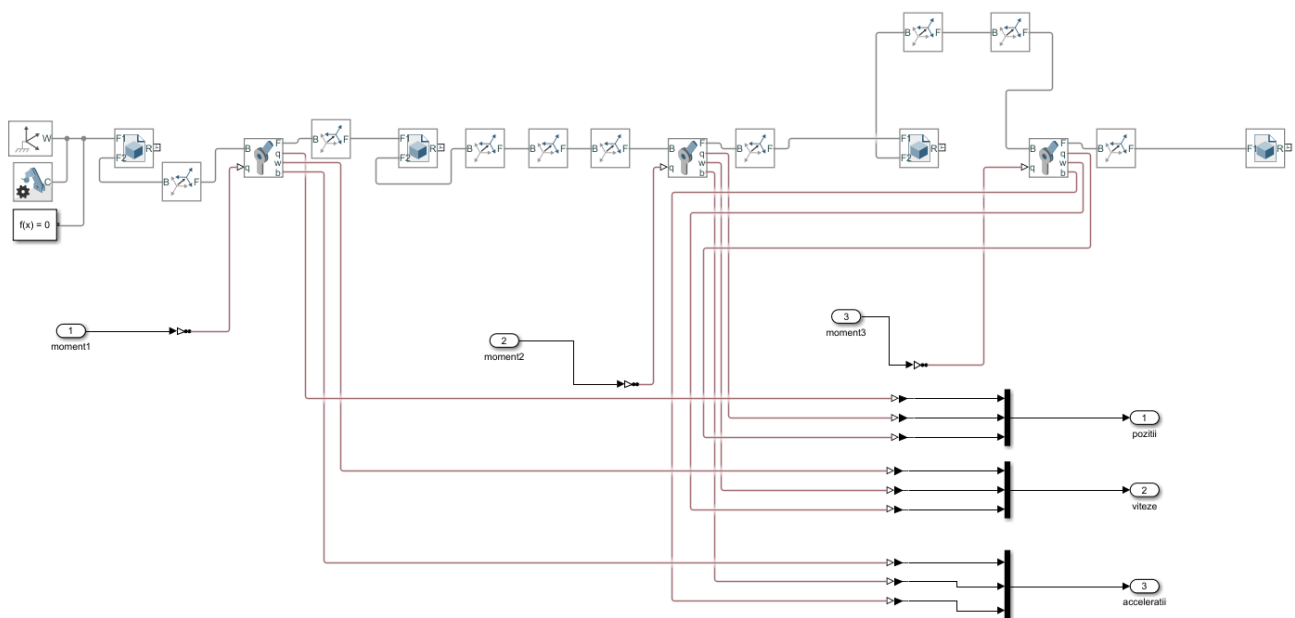


Fig 3.4.9 Model robot

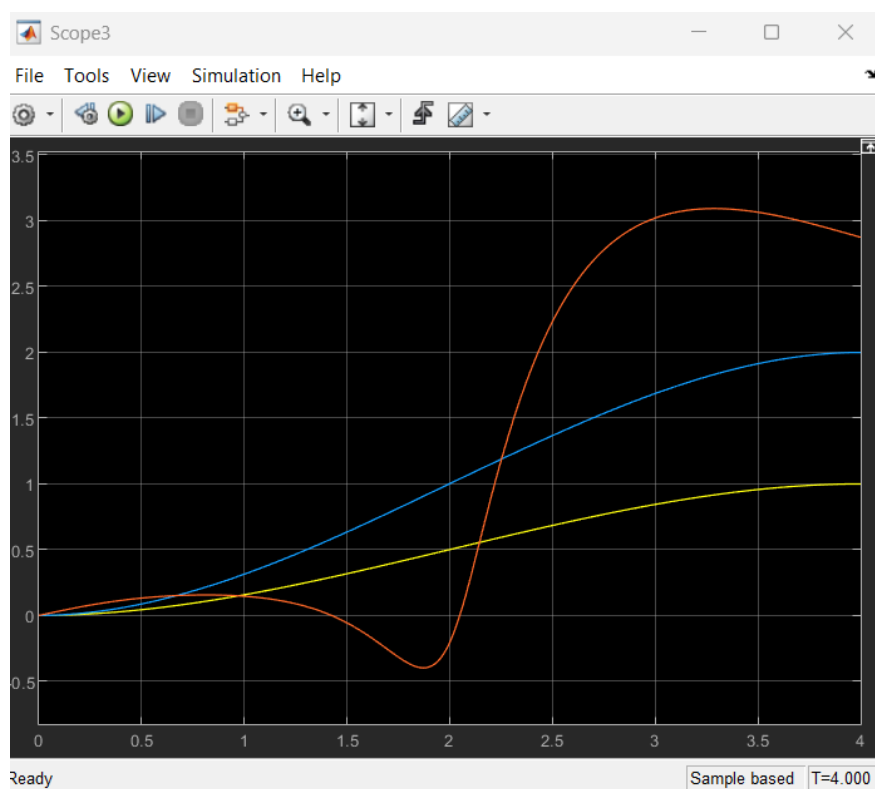


Fig 3.4.10 Pozitia cuplelor

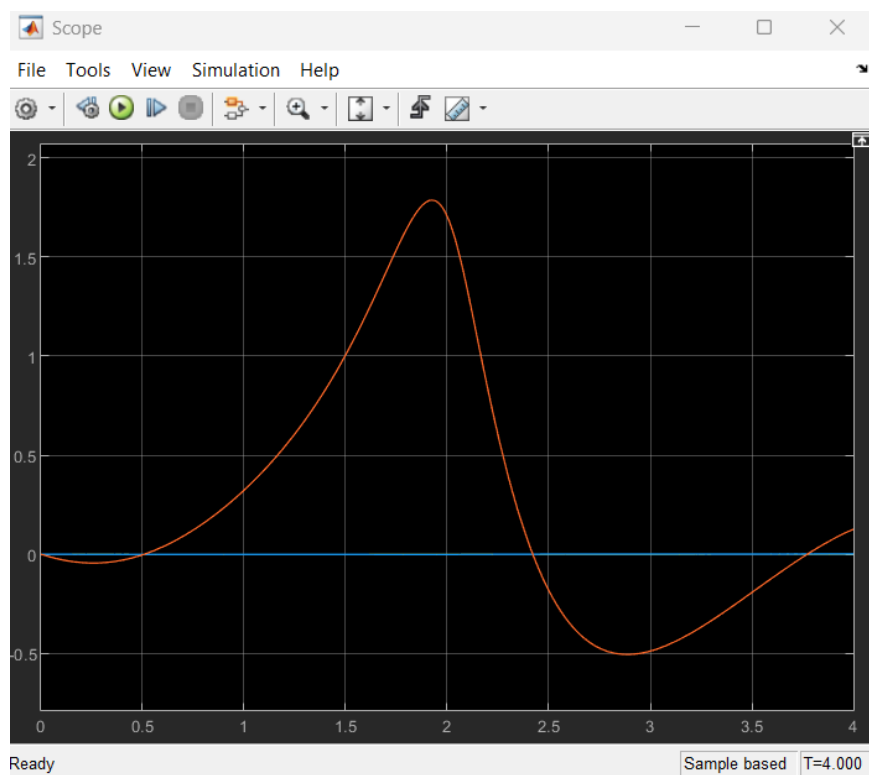


Fig 3.4.11 Eroarea pozitiei

3.5 MODELUL FANTOMA

Pentru a nu se mai folosi matricile Coriolis, centrifuga, de inertie si gravitationala, ne ajutam de modelul dinamic direct(albastru), si cel invers(rosu). Valorile la regulatorul PID sunt $K_p=10000$ si $K_i=0$. Functioneaza si daca se integreaza acceleratia din *Sum*.

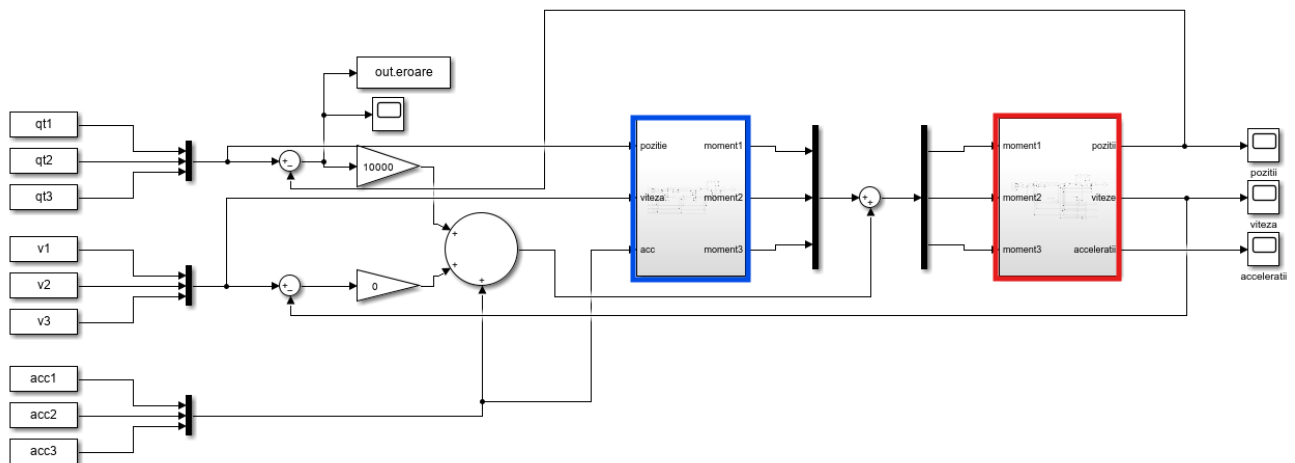


Fig 3.5.1 Schema block Simulink

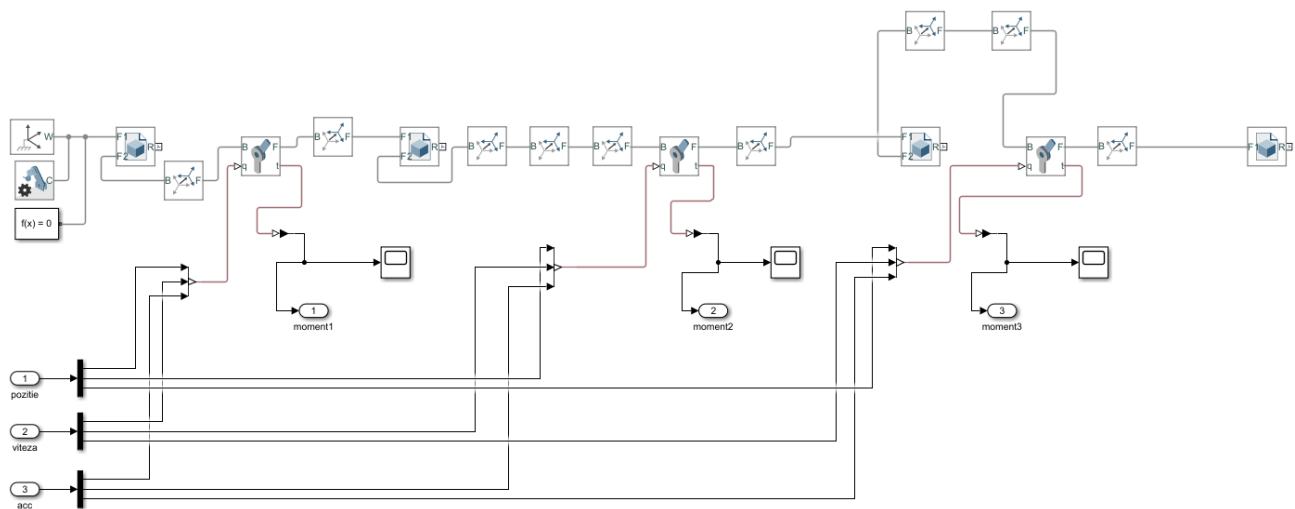


Fig 3.5.2 Modelul dinamic direct

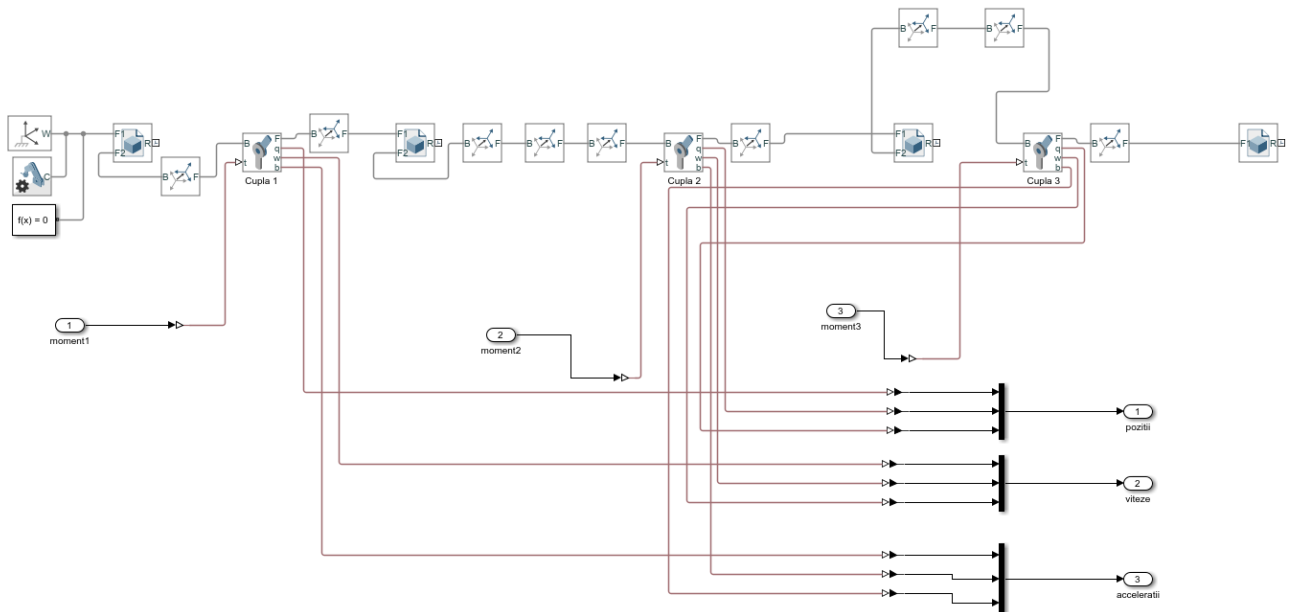


Fig 3.5.3 Modelul dinamic invers

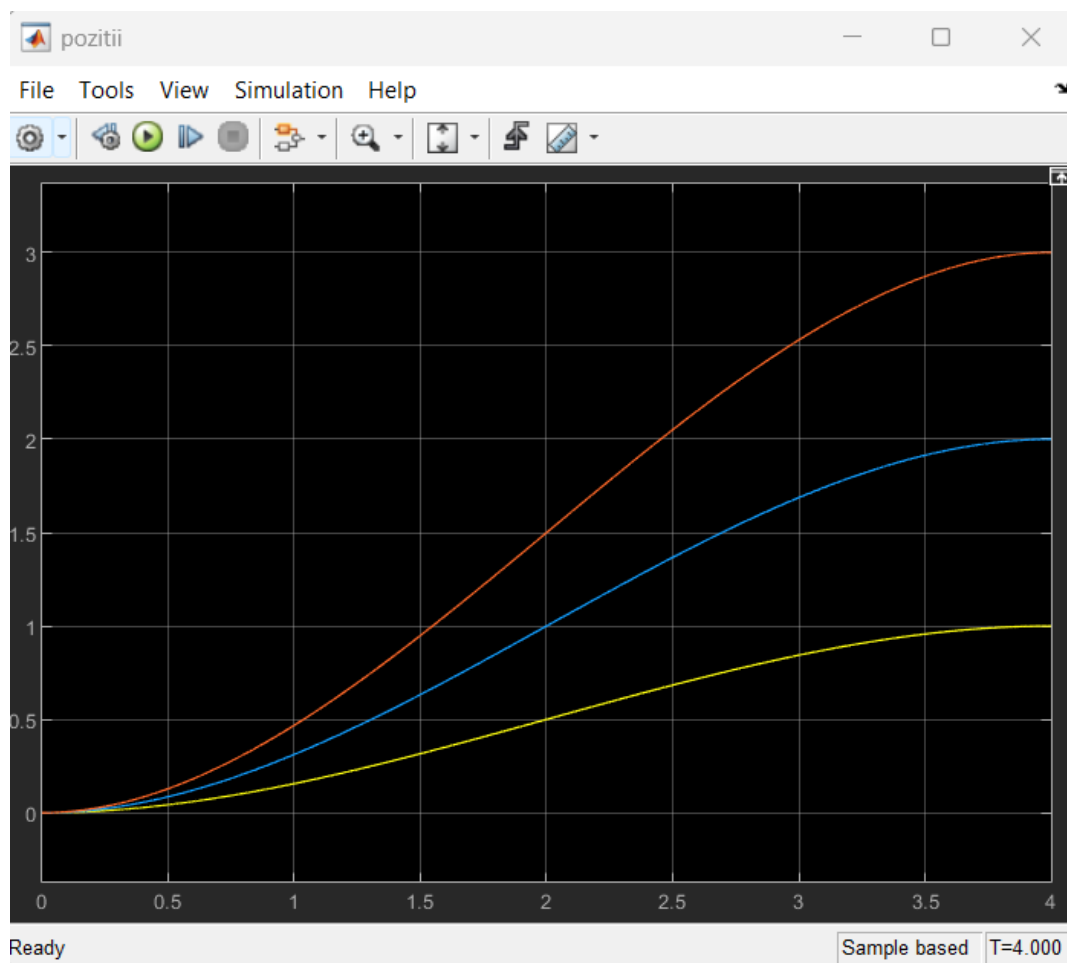


Fig 3.5.4 Pozitia cuplelor

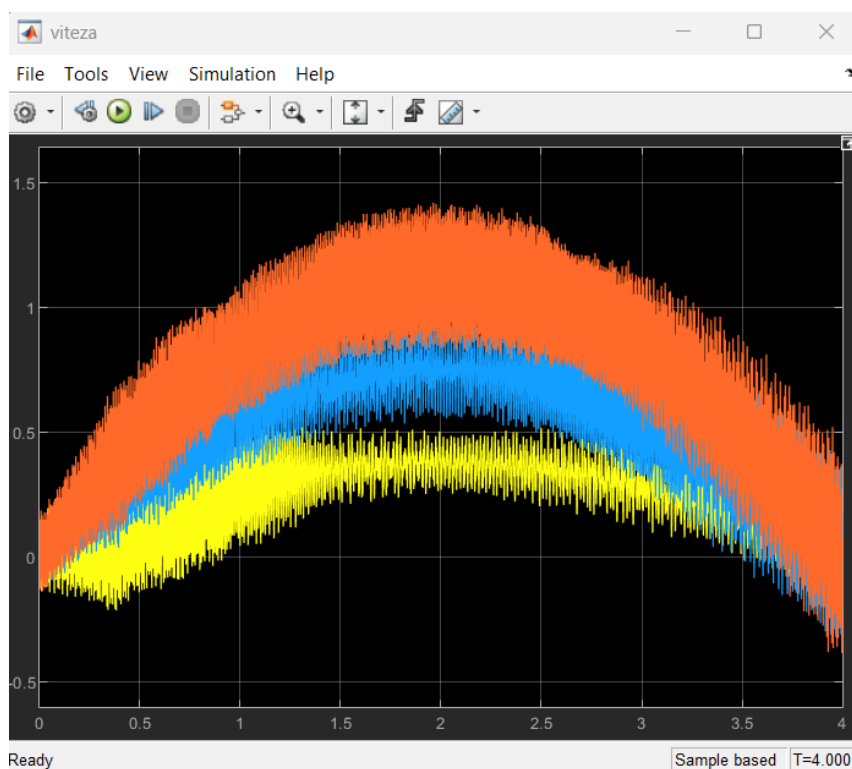


Fig 3.5.5 Viteza cuplelor

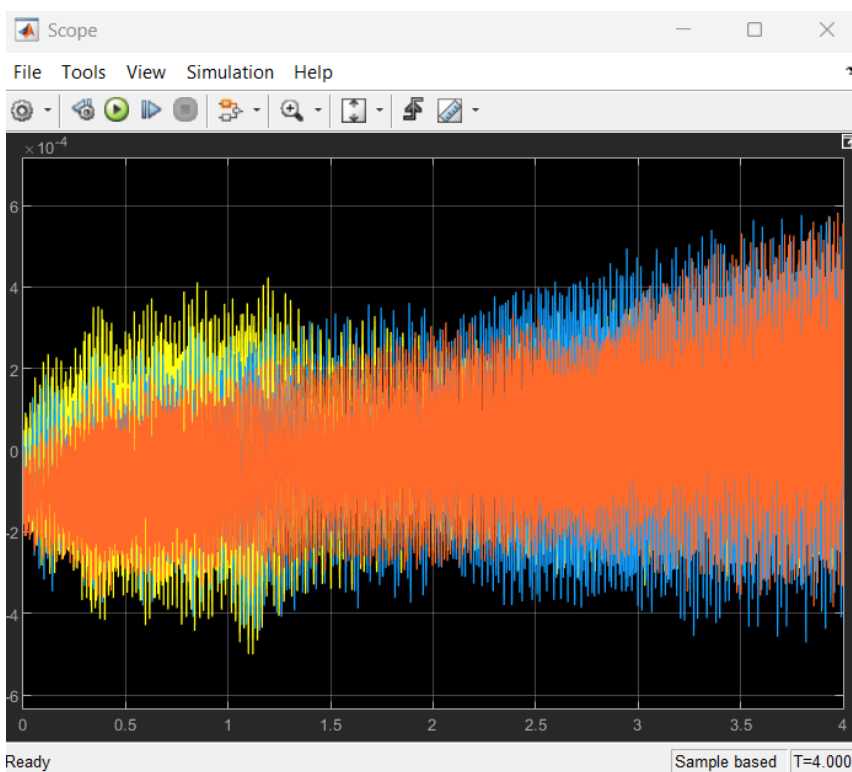


Fig 3.5.6 Eroarea pozitiilor

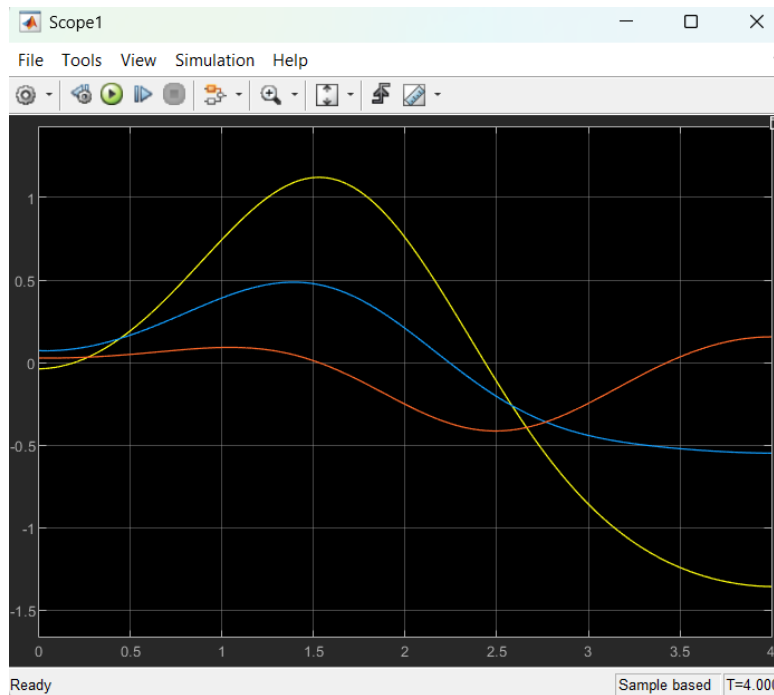


Fig 3.5.7 Momentele cuplelor

3.6 CONTROLUL IN SPATIUL CARTEZIAN

In ultima metoda exista 2 diferente majore fata de *Controlul dinamic invers*, anume:

- Introducerea Jacobianul, impreuna cu x_e , y_e , z_e (distanţa de la ultima cupla la efector).
- Parametrii de intrare (poziția, viteza și accelerația) sunt în spațiul cartezian în loc de cel articular.

Pentru a transforma parametrii în spațiul cartezian se folosește Geometria Directă.

Matricile de transformare:

Transformarea	Matricea
Rotatie in jurul axei Z	$T1 = \begin{bmatrix} \cos(q(1,:)), & -\sin(q(1,:)), & 0, & 0; \\ \sin(q(1,:)), & \cos(q(1,:)), & 0, & 0; \\ 0, & 0, & 1, & 0; \\ 0, & 0, & 0, & 1 \end{bmatrix};$
Rotatie in jurul axei Y	$T2 = \begin{bmatrix} \cos(q(2,:)), & 0, & \sin(q(2,:)), & l1; \\ 0, & 1, & 0, & 0; \\ -\sin(q(2,:)), & 0, & \cos(q(2,:)), & 0; \\ 0, & 0, & 0, & 1 \end{bmatrix};$
Rotatie in jurul axei Y	$T3 = \begin{bmatrix} \cos(q(3,:)), & 0, & \sin(q(3,:)), & l2+l3; \\ 0, & 1, & 0, & 0; \\ -\sin(q(3,:)), & 0, & \cos(q(3,:)), & 0; \\ 0, & 0, & 0, & 1 \end{bmatrix};$

```
function x = fcn(q)

l1=141 * 10 ^-3;
l2=255 * 10 ^-3;
l3=255 * 10 ^-3;

P0=[0;0;0;1];

T1 = [cos(q(1,:)), -sin(q(1,:)), 0, 0;...
      sin(q(1,:)), cos(q(1,:)), 0, 0;...
      0, 0, 1, 0;...
      0, 0, 0, 1];
%rotatie pe Z

T2 = [cos(q(2,:)), 0, sin(q(2,:)), l1;...
      0, 1, 0, 0;...
      -sin(q(2,:)), 0, cos(q(2,:)), 0;...
      0, 0, 0, 1];
%rotatie pe Y

T3 = [cos(q(3,:)), 0, sin(q(3,:)), l2+l3;...
      0, 1, 0, 0;...
      -sin(q(3,:)), 0, cos(q(3,:)), 0;...
      0, 0, 0, 1];
%rotatie pe Y

poz=T1*T2*T3*P0;

x = poz(1:3,:);
```

Fig 3.6.1 Geometria directa

Pentru viteza si acceleratie in spatiul cartezian se foloseste Jacobianul.

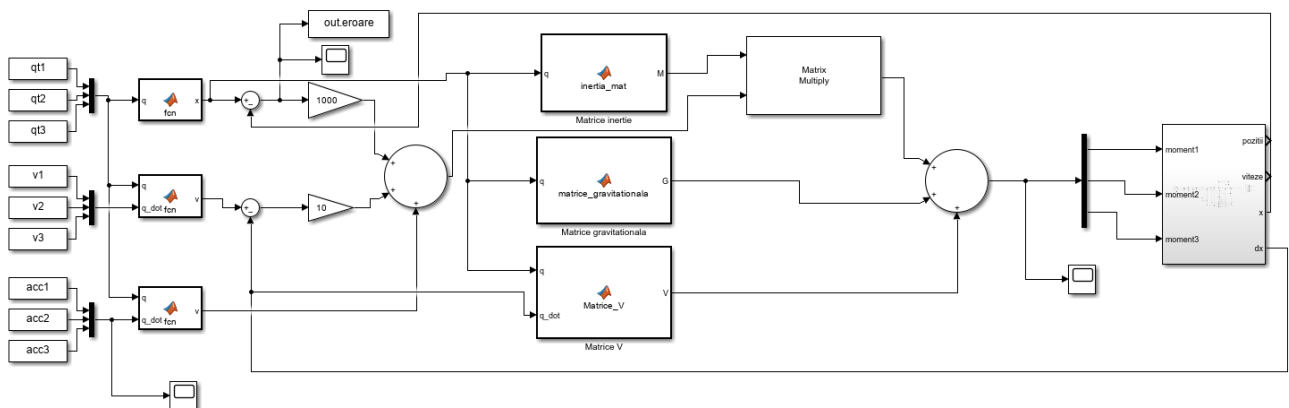


Fig 3.6.2 Schema block Simulink

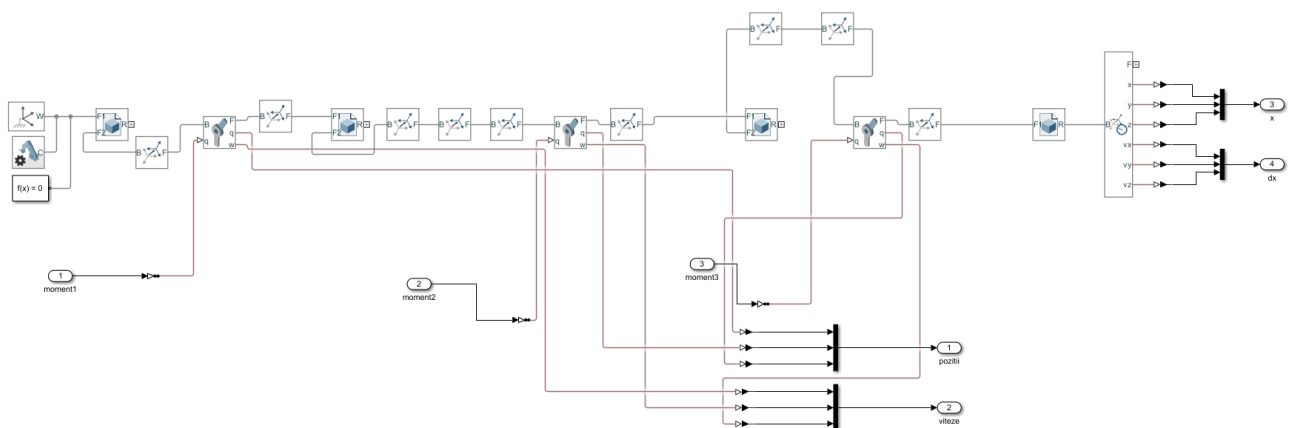


Fig 3.6.3 Model robot

4.BIBLIOGRAFIE

- Pozna, C -Dinamica robotilor
- Pozna, C -Sisteme de Conducere in Robotica